

Ottodot LMS — Homework Submission Flow

Assessment Report & Improvement Plan

Prepared by Jason Ramirez

October 8, 2025 (GMT+8)

Contents

1	Question 1: Concerns	6
1.1	Homework Core Utilities — Error Semantics, Idempotency & Retry	6
1.1.1	Critical issue 1 — Ambiguous error semantics	6
1.1.1.1	Issue.	6
1.1.1.2	Root cause.	6
1.1.1.3	Severity (rationale).	6
1.1.1.4	Evidence.	6
1.1.1.5	Representative snippet.	6
1.1.1.6	Impact.	7
1.1.1.7	Assessment.	7
1.1.2	Critical issue 2 — Session creation not idempotent	7
1.1.2.1	Issue.	7
1.1.2.2	Root cause.	7
1.1.2.3	Severity (rationale).	7
1.1.2.4	Evidence.	7
1.1.2.5	Impact.	8
1.1.2.6	Assessment.	8
1.1.3	Critical issue 3 — No retry/backoff on transient failures	8
1.1.3.1	Issue.	8
1.1.3.2	Root cause.	8
1.1.3.3	Severity (rationale).	8
1.1.3.4	Evidence.	8
1.1.3.5	Representative snippet.	8
1.1.3.6	Impact.	8
1.1.3.7	Assessment.	9
1.1.4	Critical issue 4 — Data correctness / mock leakage	9
1.1.4.1	Issue.	9
1.1.4.2	Root cause.	9
1.1.4.3	Severity (rationale).	9
1.1.4.4	Evidence.	9
1.1.4.5	Impact.	9
1.1.4.6	Assessment.	9
1.1.5	Critical issue 5 — Fragile parsing & input validation	9
1.1.5.1	Issue.	9
1.1.5.2	Root cause.	9
1.1.5.3	Severity (rationale).	9
1.1.5.4	Evidence.	10
1.1.5.5	Impact.	10

	1.1.5.6	Assessment.	10
1.1.6		Critical issue 6 — Chatterly progress writes (caller-influenced)	10
	1.1.6.1	Issue.	10
	1.1.6.2	Root cause.	10
	1.1.6.3	Severity (rationale).	10
	1.1.6.4	Evidence.	10
	1.1.6.5	Impact.	11
	1.1.6.6	Assessment.	11
1.1.7		Critical issue 7 — Inconsistent ID parsing & type safety (added)	11
	1.1.7.1	Issue.	11
	1.1.7.2	Root cause.	11
	1.1.7.3	Severity (rationale).	11
	1.1.7.4	Evidence.	11
	1.1.7.5	Impact.	11
	1.1.7.6	Assessment.	12
1.2		Assignment Taker Page — Data Loading, Feedback & Navigation	12
1.2.1		Critical issue 1 — Sequential data loading on mount	12
	1.2.1.1	Issue.	12
	1.2.1.2	Root cause.	12
	1.2.1.3	Severity (rationale).	12
	1.2.1.4	Evidence.	12
	1.2.1.5	Representative snippet.	12
	1.2.1.6	Impact.	12
	1.2.1.7	Assessment.	12
1.2.2		Critical issue 2 — Silent failures & no recovery UX	12
	1.2.2.1	Issue.	12
	1.2.2.2	Root cause.	13
	1.2.2.3	Severity (rationale).	13
	1.2.2.4	Evidence.	13
	1.2.2.5	Representative snippet.	13
	1.2.2.6	Impact.	13
	1.2.2.7	Assessment.	13
1.2.3		Critical issue 3 — Over-chattiness on navigation	13
	1.2.3.1	Issue.	13
	1.2.3.2	Root cause.	13
	1.2.3.3	Severity (rationale).	13
	1.2.3.4	Evidence.	13
	1.2.3.5	Representative snippet.	13
	1.2.3.6	Impact.	13

	1.2.3.7	Assessment.	14
1.2.4		Critical issue 4 — Short-answer mismatch	14
	1.2.4.1	Issue.	14
	1.2.4.2	Root cause.	14
	1.2.4.3	Severity (rationale).	14
	1.2.4.4	Evidence.	14
	1.2.4.5	Representative snippet.	14
	1.2.4.6	Impact.	14
	1.2.4.7	Assessment.	14
1.2.5		Critical issue 5 — Repeated auth lookup & no abort guards	14
	1.2.5.1	Issue.	14
	1.2.5.2	Root cause.	14
	1.2.5.3	Severity (rationale).	14
	1.2.5.4	Evidence.	15
	1.2.5.5	Representative snippet.	15
	1.2.5.6	Impact.	15
	1.2.5.7	Assessment.	15
1.3		ReliableImage Component — Fallbacks, Prefetching & Layout Stability	15
	1.3.1	Critical issue 1 — Conservative visibility threshold	15
	1.3.1.1	Issue.	15
	1.3.1.2	Root cause.	15
	1.3.1.3	Severity (rationale).	15
	1.3.1.4	Evidence.	15
	1.3.1.5	Impact.	16
	1.3.1.6	Assessment.	16
	1.3.2	Critical issue 2 — Fallback churn & aggressive URL cleaning	16
	1.3.2.1	Issue.	16
	1.3.2.2	Root cause.	16
	1.3.2.3	Severity (rationale).	16
	1.3.2.4	Evidence.	16
	1.3.2.5	Impact.	17
	1.3.2.6	Assessment.	17
	1.3.3	Critical issue 3 — No prefetch/staging coordination	17
	1.3.3.1	Issue.	17
	1.3.3.2	Root cause.	17
	1.3.3.3	Severity (rationale).	17
	1.3.3.4	Evidence.	18
	1.3.3.5	Impact.	18
	1.3.3.6	Assessment.	18

1.3.4	Critical issue 4 — Layout stability & observability	18
1.3.4.1	Issue.	18
1.3.4.2	Root cause.	18
1.3.4.3	Severity (rationale).	18
1.3.4.4	Evidence.	18
1.3.4.5	Impact.	19
1.3.4.6	Assessment.	19
2	Question 2: One-Month Roadmap	19
2.1	Week 1: Stabilize & Feedback	19
2.2	Week 2: Reliability Patterns & Perf Hygiene	20
2.3	Weeks 3–4: Architectural Enhancements	22
3	Question 3: Quick Wins	23
3.1	Quick Win 1 — Parallelize mount fetches	23
3.2	Quick Win 2 — Visible save failure with one-tap retry	24
3.3	Quick Win 3 — Image reliability nudge	24

1 Question 1: Concerns

1.1 Homework Core Utilities — Error Semantics, Idempotency & Retry

lib/homeworkUtils.ts

Severity rationale rubric. We assign severity by combining user impact (data loss or block), frequency (how often it occurs under the mock’s 500–3000 ms latency and ~10% failure rate), recovery cost (user and system effort to recover), and detectability (how obvious it is to users and telemetry). High = user-blocking or data-loss perception with high frequency and poor recovery; Medium–High = frequent UX degradation with some recovery; Medium = noticeable but bounded risk or easy recovery.

1.1.1 Critical issue 1 — Ambiguous error semantics

1.1.1.1 Issue. Functions return `false`/`null` on failure, hiding whether the error is transient or fatal and providing no machine-readable guidance to the UI.

1.1.1.2 Root cause. No structured error objects (e.g., `{code, message, retrieable}`); errors are only logged to console.

1.1.1.3 Severity (rationale). **High** — Impact: users experience silent save/submit failures with no actionable recovery; Frequency: any transient blip triggers it; Recovery cost: high because callers cannot distinguish retryable vs. fatal; Detectability: low—console-only logs provide no in-product signal.

1.1.1.4 Evidence.

- `getHomeworkAssignment` — returns `null` on error.
- `saveHomeworkResponse` — returns `false` on error.
- `submitHomeworkAssignment` — returns `false` on error.
- `updateHomeworkSessionProgress` — returns `false` on error.

1.1.1.5 Representative snippet.

```
1 export async function saveHomeworkResponse(...) : Promise<boolean> {
2   try {
3     ...
4     const { error } = await (mockSupabase.from('homework_responses') as any)
5       .upsert(responseData, { onConflict: 'student_id,question_id,assignment_id' });
6     if (error) {
7       console.error('Error saving homework response:', error);
```

```

8     return false; // ambiguous for callers
9 }
10 return true;
11 } catch (error) {
12     console.error('Error in saveHomeworkResponse:', error);
13     return false; // also ambiguous
14 }
15 }

```

1.1.1.6 Impact. Callers cannot choose user-facing copy (e.g., “try again” vs. “contact support”), cannot implement targeted retry, and analytics cannot distinguish transient classes.

1.1.1.7 Assessment. Standardize on structured errors `{code, message, retrieable}`; reserve boolean for success only.

1.1.2 Critical issue 2 — Session creation not idempotent

1.1.2.1 Issue. Duplicate sessions can be created during transient errors/races, corrupting progress semantics.

1.1.2.2 Root cause. Fetch-then-create flow without unique constraint semantics or conflict handling on `{student_id, assignment_id}`.

1.1.2.3 Severity (rationale). **High** — Impact: duplicated sessions break resume state and risk progress loss; Frequency: likely whenever network flaps or multiple opens occur; Recovery cost: high—manual data repair or user confusion; Detectability: medium—duplicates may only surface later when progress appears inconsistent.

1.1.2.4 Evidence. The file-level note states: “Creates new session even on temporary network failures.” The implementation:

```

1 export async function getOrCreateHomeworkSession(studentId: string, assignmentId: number
  | string) {
2     const { data: existingSession, error: sessionError } =
3         await (mockSupabase.from('homework_sessions') as any)
4             .select('*').eq('student_id', studentId).eq('assignment_id', assignmentIdNum).
5             maybeSingle();
6
7     if (existingSession && !sessionError) {
8         await (mockSupabase.from('homework_sessions') as any)
9             .update({ last_activity: new Date().toISOString() }).eq('id', existingSession.id);
10        return existingSession;
11    }
12 }

```

```

11  const { data: newSession } = await (mockSupabase.from('homework_sessions') as any)
12    .insert({ student_id: studentId, assignment_id: assignmentIdNum, ... }).select().
    single();
13  return newSession;
14  }

```

1.1.2.5 Impact. Multiple sessions per user/assignment, broken resume behavior, inconsistent `current_question_index`.

1.1.2.6 Assessment. Enforce uniqueness on `{student_id, assignment_id}`; on conflict, re-select and return the existing row. Surface a structured `SESSION_CONFLICT` code if needed.

1.1.3 Critical issue 3 — No retry/backoff on transient failures

1.1.3.1 Issue. Single-attempt reads/writes elevate transient blips into user-facing failures.

1.1.3.2 Root cause. No bounded retry with backoff+jitter for idempotent operations.

1.1.3.3 Severity (rationale). **High** — Impact: saves/submissions fail unnecessarily; Frequency: high under $\sim 10\%$ induced failure; Recovery cost: medium to high—users must retry manually; Detectability: low to medium—failures often look random without clear messaging.

1.1.3.4 Evidence.

- `saveHomeworkResponse` (single upsert, no retry).
- `updateHomeworkSessionProgress` (single update, no retry).
- `submitHomeworkAssignment` (single upsert, no retry; session completion error only logged).

1.1.3.5 Representative snippet.

```

1  const { error } = await (mockSupabase.from('homework_sessions') as any)
2    .update(updateData).eq('id', sessionId);
3  if (error) {
4    console.error('Error updating homework session:', error);
5    return false; // no retry, no classification
6  }

```

1.1.3.6 Impact. With 10% simulated failures, the effective success rate is materially worse than necessary; users see “random” failures.

1.1.3.7 Assessment. Add 2–3 retries with exponential backoff and jitter for idempotent paths; on exhaustion, return structured error with `retriable=true/false`.

1.1.4 Critical issue 4 — Data correctness / mock leakage

1.1.4.1 Issue. The function returns `mockAssignment` regardless of the actual `assignments` query result, hiding production behavior and risking stale content.

1.1.4.2 Root cause. Convenience return of mock data post-query.

1.1.4.3 Severity (rationale). **High** — Impact: incorrect questions/content shown to students; Frequency: systematic—always returns mock; Recovery cost: high—hard to diagnose inconsistencies; Detectability: low—UI looks “fine” but is wrong.

1.1.4.4 Evidence.

```
1 const { data: assignment } = await (mockSupabase.from('assignments') as any)
2   .select().eq('id', assignmentIdNum).single();
3 // ...
4 return mockAssignment; // returned regardless
```

1.1.4.5 Impact. UI displays mock content even if DB row differs or is missing; undermines correctness and confuses debugging.

1.1.4.6 Assessment. Gate the mock behind an explicit flag/env or return the queried data by default; preserve diagnostic errors.

1.1.5 Critical issue 5 — Fragile parsing & input validation

1.1.5.1 Issue. Tickbox grading assumes JSON-encoded arrays; parsing errors collapse to `false` results without granular context.

1.1.5.2 Root cause. Unvalidated types for both student and correct answers; `JSON.parse` used without schema checks.

1.1.5.3 Severity (rationale). **Medium–High** — Impact: misgrading and bad feedback; Frequency: moderate whenever encoding differs; Recovery cost: medium—requires user education or schema validation; Detectability: low—appears as “wrong” grading.

1.1.5.4 Evidence.

```
1 } else if (question.question_type === 'checkbox') {
2   try {
3     const studentAnswers = JSON.parse(responseText);
4     const correctAnswers = Array.isArray(question.correct_answer)
5       ? question.correct_answer
6       : typeof question.correct_answer === 'string'
7         ? JSON.parse(question.correct_answer || '[]')
8         : [];
9     const studentSet = new Set(studentAnswers);
10    const correctSet = new Set(correctAnswers);
11    const hasAllCorrect = correctAnswers.every((a: string) => studentSet.has(a));
12    const hasNoIncorrect = studentAnswers.every((a: string) => correctSet.has(a));
13    isCorrect = hasAllCorrect && hasNoIncorrect;
14  } catch (e) {
15    console.error('Error auto-grading checkbox question:', e);
16    isCorrect = false;
17  }
18 }
```

1.1.5.5 Impact. Misgrading and unhelpful error signals; caller cannot advise the user to correct malformed input.

1.1.5.6 Assessment. Validate payloads before parse (array shape/type); on failure, return a structured error (e.g., `GRADE_INPUT_INVALID`).

1.1.6 Critical issue 6 — Chatty progress writes (caller-influenced)

1.1.6.1 Issue. Each call updates timestamps and writes immediately; when triggered by per-click navigation, this becomes chatty and couples UX to network.

1.1.6.2 Root cause. No debounce/coalescing or “low-priority” mode; always updates `last_activity`.

1.1.6.3 Severity (rationale). Medium–High — Impact: sluggish navigation and extra failures; Frequency: high—called on every question change; Recovery cost: medium—requires coalescing or queue; Detectability: medium—visible lag but non-blocking.

1.1.6.4 Evidence.

```

1 export async function updateHomeworkSessionProgress(sessionId: string, idx: number,
    isCompleted = false) {
2   const updateData: any = { current_question_index: idx, last_activity: new Date().
    toISOString() };
3   if (isCompleted) { updateData.is_completed = true; updateData.completed_at = new Date().
    toISOString(); }
4   const { error } = await (mockSupabase.from('homework_sessions') as any)
    .update(updateData).eq('id', sessionId);
5   if (error) return false;
6   return true;
7 }
8

```

1.1.6.5 Impact. Sluggish navigation feel, unnecessary load, more chances for transient failures to surface.

1.1.6.6 Assessment. Provide debounce-friendly API and allow fire-and-forget updates with a final “flush latest” on submit/unload; add bounded retry.

1.1.7 Critical issue 7 — Inconsistent ID parsing & type safety (added)

1.1.7.1 Issue. Frequent `number` | `string` inputs with ad-hoc `parseInt` cause repeated parsing logic and NaN checks.

1.1.7.2 Root cause. Lack of typed `AssignmentId/SessionId` and central coercion.

1.1.7.3 Severity (rationale). **Medium** — Impact: noisy boilerplate and inconsistent errors; Frequency: moderate across several helpers; Recovery cost: low—central helper resolves; Detectability: medium—present in many paths but not user-blocking.

1.1.7.4 Evidence.

```

1 const assignmentIdNum = typeof assignmentId === 'string' ? parseInt(assignmentId, 10) :
    assignmentId;
2 if (isNaN(assignmentIdNum)) {
3   console.error('Invalid assignment ID:', assignmentId);
4   return null; // or false elsewhere
5 }

```

1.1.7.5 Impact. Boilerplate parsing across functions; inconsistent error semantics when parse fails.

1.1.7.6 Assessment. Centralize coercion into a helper (e.g., `coerceAssignmentId()`) that returns `{value, error}`; enforce typed IDs at call sites to reduce ambiguity.

1.2 Assignment Taker Page — Data Loading, Feedback & Navigation

pages/student/assignment/[id].tsx

1.2.1 Critical issue 1 — Sequential data loading on mount

1.2.1.1 Issue. Sequential awaits produce 5–10 s time-to-visible.

1.2.1.2 Root cause. Serial `useEffect` chain: `auth` → `access` → `assignment` → `session` → responses.

1.2.1.3 Severity (rationale). **High** — Impact: blocks first render and defers any interaction; Frequency: every page load; Recovery cost: high because users can only wait or reload; Detectability: high via user perception and network waterfall.

1.2.1.4 Evidence. Lines 40–101.

1.2.1.5 Representative snippet.

```
1 useEffect(() => {  
2   const loadAssignmentData = async () => {  
3     const { data: { user } } = await supabase.auth.getUser();  
4     const canAccess = await canStudentAccessAssignment(user.id, id);  
5     const assignmentData = await getHomeworkAssignment(id);  
6     const sessionData = await getOrCreateHomeworkSession(user.id, id);  
7     const responses = await getStudentHomeworkResponsesMap(user.id, id);  
8   };  
9   loadAssignmentData();  
10 }, [id, router]);
```

1.2.1.6 Impact. Slow first paint; abandonment risk.

1.2.1.7 Assessment. Parallelize post-auth with `Promise.allSettled`; progressive rendering; add cancel guards.

1.2.2 Critical issue 2 — Silent failures & no recovery UX

1.2.2.1 Issue. Errors logged only; no banners/toasts/retry.

1.2.2.2 Root cause. Optimistic UI without surfacing failure; coarse `false/null` from `lib`.

1.2.2.3 Severity (rationale). **High** — Impact: saves/submission can fail without the student knowing; Frequency: common under ~10% induced failure; Recovery cost: high because the only recourse is to retype or resubmit; Detectability: low in-product, high only in console.

1.2.2.4 Evidence. `handleAnswer`, `handleTickboxAnswer`, `handleSubmit`.

1.2.2.5 Representative snippet.

```
1 const success = await saveHomeworkResponse(user.id, question.id, assignment.id, answer,
   question, session.id);
2 if (!success) {
3   console.warn('Failed to save response, continuing with local state');
4 }
```

1.2.2.6 Impact. Users think work saved when it didn't.

1.2.2.7 Assessment. Show actionable errors; offer retry; use structured errors from `lib`.

1.2.3 Critical issue 3 — Over-chattiness on navigation

1.2.3.1 Issue. Awaiting progress write on each Previous/Next click.

1.2.3.2 Root cause. Blocking network call in navigation handlers.

1.2.3.3 Severity (rationale). **Medium–High** — Impact: visible lag on each step; Frequency: every navigation; Recovery cost: medium as debouncing can mitigate; Detectability: high to users as UI slowness.

1.2.3.4 Evidence. `handlePrevious`/`handleNext`.

1.2.3.5 Representative snippet.

```
1 const newIndex = currentQuestion + 1;
2 setCurrentQuestion(newIndex);
3 await updateHomeworkSessionProgress(session.id, newIndex);
```

1.2.3.6 Impact. Sluggish switching.

1.2.3.7 Assessment. Debounce/coalesce; non-blocking navigation; small syncing indicator.

1.2.4 Critical issue 4 — Short-answer mismatch

1.2.4.1 Issue. String path routed through `getOptionText` expecting option-like object.

1.2.4.2 Root cause. Shared handler across different input shapes.

1.2.4.3 Severity (rationale). **Medium** — Impact: can store malformed values or lose fidelity; Frequency: when short-answer used; Recovery cost: medium; Detectability: medium in data checks, low to end users.

1.2.4.4 Evidence. `renderQuestion` input $\rightarrow$ `handleAnswer`.

1.2.4.5 Representative snippet.

```
1 <input
2   type="text"
3   value={answers[question.id] || ''}
4   onChange={(e) => handleAnswer(e.target.value)}
5 />
6
7 const handleAnswer = async (option: any) => {
8   const answer = getOptionText(option);
9 };
```

1.2.4.6 Impact. Incorrect persistence.

1.2.4.7 Assessment. Separate string path; bypass `getOptionText`.

1.2.5 Critical issue 5 — Repeated auth lookup & no abort guards

1.2.5.1 Issue. Extra latency; state-after-unmount risk.

1.2.5.2 Root cause. Fetching user in multiple handlers; missing `AbortController`/`isMounted` guards.

1.2.5.3 Severity (rationale). **Medium** — Impact: wasted requests and potential memory/state warnings; Frequency: across multiple handlers and route changes; Recovery cost: low to medium; Detectability: medium in logs and perf traces.

1.2.5.4 Evidence. Handlers and mount effect.

1.2.5.5 Representative snippet.

```
1 const { data: { user } } = await supabase.auth.getUser();
```

1.2.5.6 Impact. Wasted requests; warnings/flicker on route change.

1.2.5.7 Assessment. Cache `user.id` once; cancel in-flight on unmount.

1.3 ReliableImage Component — Fallbacks, Prefetching & Layout Stability

components/ReliableImage.tsx

Severity rationale rubric. Severity reflects user impact (blocking vs. cosmetic), frequency (how often it occurs across typical sessions), recovery cost (how hard it is to recover in product), and detectability (how visible it is to users/telemetry). High = user-blocking or data-loss perception and frequent; Medium–High = frequent UX degradation with some recovery; Medium = noticeable but bounded; Low–Medium = minor or infrequent but worth attention.

1.3.1 Critical issue 1 — Conservative visibility threshold

1.3.1.1 Issue. The 50px `rootMargin` can start loads too late for fast scrolls or quick navigation, causing images to appear after the user arrives.

1.3.1.2 Root cause. `IntersectionObserver` is configured with a small threshold and is attached after a zero-delay timeout.

1.3.1.3 Severity (rationale). **Medium** — Impact: perceived lateness but not blocking; Frequency: common on laptops and fast scrolling; Recovery cost: low (config tweak); Detectability: high to users as a late reveal.

1.3.1.4 Evidence.

```
1 const observer = new IntersectionObserver(  
2   ([entry]) => {  
3     if (entry.isIntersecting && mountedRef.current) {  
4       setIsVisible(true);  
5       observer.disconnect();  
6     }  
  })
```

```

7   },
8   { rootMargin: '50px' }
9 );
10
11 const timeoutId = setTimeout(() => {
12   if (imgRef.current && mountedRef.current) {
13     observer.observe(imgRef.current);
14   }
15 }, 0);

```

1.3.1.5 Impact. Images may initiate loading only when nearly visible, missing an opportunity to warm up earlier.

1.3.1.6 Assessment. Increase `rootMargin` to 200–400 px for upcoming content; consider immediate observer attachment when the ref is available.

1.3.2 Critical issue 2 — Fallback churn & aggressive URL cleaning

1.3.2.1 Issue. Multi-step fallbacks and aggressive URL cleaning can add extra round-trips and mask upstream content issues.

1.3.2.2 Root cause. The component retries with a cleaned URL before moving to placeholder and finally text fallback.

1.3.2.3 Severity (rationale). **Low–Medium** — Impact: small latency tax and observability dilution; Frequency: whenever URLs are imperfect; Recovery cost: low; Detectability: medium in DevTools and logs.

1.3.2.4 Evidence.

```

1  const validateAndCleanUrl = useCallback((url: string): string | null => {
2    const trimmedUrl = url.trim();
3    if (
4      trimmedUrl.startsWith('https://') || trimmedUrl.startsWith('http://') ||
5      trimmedUrl.startsWith('/') || trimmedUrl.startsWith('data:') ||
6      trimmedUrl.startsWith('./') || trimmedUrl.startsWith('../')
7    ) {
8      return trimmedUrl;
9    }
10   if (trimmedUrl.includes('/://') && !trimmedUrl.startsWith('http')) {
11     return `https://${trimmedUrl.split('/://')[1]}`;
12   }

```



```

13   if (trimmedUrl.includes('.') && !trimmedUrl.includes(' ') && !trimmedUrl.includes('\n'))
14       ) {
15       return `https://${trimmedUrl}`;
16   }
17   return null;
18 }, []);
19
20 const handleError = useCallback(() => {
21   if (fallbackLevel === 0 && src) {
22     const cleanedUrl = src.trim().replace(/\s+/g, '').replace(/\n/g, '');
23     if (cleanedUrl !== currentSrc && validateAndCleanUrl(cleanedUrl)) {
24       setCurrentSrc(cleanedUrl);
25       setFallbackLevel(1);
26       return;
27     }
28   }
29   if (fallbackLevel < 2) {
30     setCurrentSrc(PHOTOHOLDER_IMAGE_DATA_URI);
31     setFallbackLevel(2);
32     return;
33   }
34   setHasError(true);
35   setFallbackLevel(3);
36   onError?.();
37 }, [currentSrc, fallbackLevel, src, onError, validateAndCleanUrl]);

```

1.3.2.5 Impact. Adds latency when a bad URL is retried and may hide content pipeline defects if placeholders silently cover them.

1.3.2.6 Assessment. Emit an analytics event on each fallback step; consider limiting cleaning to a narrower, safer subset and surfacing a content QA signal upstream.

1.3.3 Critical issue 3 — No prefetch/staging coordination

1.3.3.1 Issue. The component does not coordinate with the page to prefetch next-question images or to stage multiple images by priority.

1.3.3.2 Root cause. Behavior is deliberately component-scoped; there is no contract for prefetch or srcset/sizes.

1.3.3.3 Severity (rationale). **Medium** — Impact: slower perceived loads on question transitions; Frequency: frequent in multi-question flows; Recovery cost: moderate (page-level changes); Detectability: high in waterfalls and user reports of placeholder flashes.

1.3.3.4 Evidence.

```
1 return (  
2   <img  
3     ref={imgRef}  
4     src={isVisible ? (currentSrc || PLACEHOLDER_IMAGE_DATA_URI) :  
5       PLACEHOLDER_IMAGE_DATA_URI}  
6     alt={alt}  
7     className={className}  
8     style={style}  
9     onLoad={handleLoad}  
10    onError={handleError}  
11    loading={priority ? 'eager' : 'lazy'}  
12    decoding="async"  
13  />  
14 );
```

1.3.3.5 Impact. When navigating rapidly, images for the next screen are not warmed; bandwidth contention is uncoordinated.

1.3.3.6 Assessment. Add an optional `prefetchHint` or expose events so the page can prefetch next-question hero images; consider `srcset/sizes` to right-size payloads.

1.3.4 Critical issue 4 — Layout stability & observability

1.3.4.1 Issue. Potential layout shifts without explicit dimensions; observability relies on console logs only.

1.3.4.2 Root cause. The `img` tag lacks `width/height/style.aspectRatio`; logging hooks do not emit to analytics by default.

1.3.4.3 Severity (rationale). **Low–Medium** — Impact: visual jitter but non-blocking; Frequency: varies by caller styles; Recovery cost: low; Detectability: high visually, low in telemetry.

1.3.4.4 Evidence.

```
1 console.log('ReliableImage Error:', { src, currentSrc, fallbackLevel, alt });  
2 console.log('ReliableImage Load Success:', { src, currentSrc, alt });  
3  
4 return (  
5   <img
```

```

6   ref={imgRef}
7   src={...}
8   alt={alt}
9   className={className}
10  style={style}
11  onLoad={handleLoad}
12  onError={handleError}
13  loading={priority ? 'eager' : 'lazy'}
14  decoding="async"
15  />
16 );

```

1.3.4.5 Impact. CLS risk on first render; lack of metrics hinders diagnosing image reliability and fallback rates.

1.3.4.6 Assessment. Encourage width/height or an `aspect-ratio` style at call sites; optionally add an `onStatus` callback to surface analytics events for `{loaded, placeholder, text_fallback}`.

2 Question 2: One-Month Roadmap

2.1 Week 1: Stabilize & Feedback

- **Parallelize mount fetches (post-auth).**
 - *What.* After obtaining `user.id` once, issue access, assignment, session, and responses via `Promise.allSettled`, render partials, and guard against stale updates.
 - *Why now.* Biggest TTV win with low risk; removes serialized latency.
 - *Impact.* First question visible in ~2–3s under mock worst-case; fewer abandonment reports.

```

1  const { data: { user } } = await supabase.auth.getUser();
2  const [accessR, assignR, sessionR, responsesR] = await Promise.allSettled([
3    canStudentAccessAssignment(user.id, id),
4    getHomeworkAssignment(id),
5    getOrCreateHomeworkSession(user.id, id),
6    getStudentHomeworkResponsesMap(user.id, id),
7  ]);
8  // Progressive render using whichever are fulfilled; show inline notices for rejected.

```

- **Visible error + retry for saves/submission.**
 - *What.* Replace console-only handling with a non-blocking banner and a *Retry* action that re-sends a captured payload; display specific messages when errors are retrievable vs. fatal.

- *Why now.* Eliminates silent failures quickly; restores trust.
- *Impact.* 0% silent save failures; clear user guidance on next steps.

```

1 const pending = { studentId: user.id, qid: question.id, payload: answer };
2 const ok = await saveHomeworkResponse(user.id, question.id, assignment.id, answer,
    question, session.id);
3 if (!ok) {
4   setToast({ kind: 'error', text: 'Save failed', retry: () => retrySave(pending) });
5 }

```

- **Short-answer correctness.**

- *What.* Treat text input as plain string; bypass `getOptionText`.
- *Why now.* Low-effort bug fix preventing malformed persistence.
- *Impact.* Correct storage and grading for short answers.

```

1 <input value={answers[q.id] || ''} onChange={(e) => handleShortAnswer(e.target.value)}
  />

```

- **Image reliability tweaks.**

- *What.* Increase `rootMargin` to 300–400 px in `ReliableImage` call context; ensure width/height or aspect-ratio at call sites to reduce CLS.
- *Why now.* Immediate perceived-speed lift; minimal risk.
- *Impact.* Fewer placeholder flashes; earlier image starts.

```

1 <ReliableImage src={url} alt="Question visual" className="rounded-xl" style={{
    aspectRatio: '4 / 3' }} />

```

2.2 Week 2: Reliability Patterns & Perf Hygiene

- **Bounded retry with backoff & jitter.**

- *What.* Wrap `saveHomeworkResponse`, `updateHomeworkSessionProgress`, and submission in 2–3 retries with exponential backoff and jitter; propagate structured errors.
- *Why then.* Builds on Week 1 feedback UI; reduces random failure feel.
- *Impact.* Higher effective success rates under transient mock failures.

```

1 async function withRetry(fn, { tries = 3, base = 250 }) {
2   let attempt = 0;
3   while (attempt < tries) {
4     const res = await fn();
5     if (res === true) return true;
6     const t = base * 2 ** attempt + Math.floor(Math.random() * 100);
7     await new Promise(r => setTimeout(r, t));
8     attempt++;
9   }
10  return false;
11 }

```

- **Coalesce progress writes.**

- *What.* Debounce session progress updates and only send the latest index; do not block navigation.
- *Why then.* Requires minor refactor around navigation; safe after Week 1.
- *Impact.* Snappier question switching; fewer network calls.

```

1 let flushTimer;
2 function setIndexAndQueueSave(idx) {
3   setCurrentQuestion(idx);
4   clearTimeout(flushTimer);
5   flushTimer = setTimeout(() => updateHomeworkSessionProgress(session.id, idx), 350);
6 }

```

- **Single user lookup.**

- *What.* Cache `user.id` at mount and pass via context/ref to handlers.
- *Why then.* Easy latency win and removes duplication.
- *Impact.* Fewer auth calls and cleaner handlers.

```

1 const [uid, setUid] = useState<string | null>(null);
2 useEffect(() => { supabase.auth.getUser().then(({ data: { user } }) => setUid(user?.id ?? null)); }, []);

```

- **Image prefetch & status analytics.**

- *What.* Prefetch the next question’s primary image once the current question renders; add an optional `onStatus` to `ReliableImage` to track fallbacks.
- *Why then.* Complements Week 1 image changes; introduces observability.
- *Impact.* Higher cache hit-rate on next navigation; measurable reliability.

```

1 useEffect(() => {
2   const next = assignment?.questions[currentQuestion + 1];
3   const url = next?.image_url || next?.image_urls?.[0];
4   if (url) { const img = new Image(); img.fetchPriority = 'low'; img.src = url; }
5 }, [assignment, currentQuestion]);

```

2.3 Weeks 3–4: Architectural Enhancements

- **Lightweight fetch manager.**

- *What.* Centralize keyed requests with dedupe, in-flight sharing, small in-memory cache, and `AbortController` cancellation; prevent state updates after unmount.
- *Why now.* Requires broader touchpoints; safer after Week 2.
- *Impact.* Fewer redundant calls; safer navigation; better responsiveness.

```

1 const inflight = new Map<string, {p: Promise<any>, abort: AbortController}>();
2 function fetchKeyed(key: string, fn: (signal: AbortSignal) => Promise<any>) {
3   if (inflight.has(key)) return inflight.get(key).p;
4   const abort = new AbortController();
5   const p = fn(abort.signal).finally(() => inflight.delete(key));
6   inflight.set(key, { p, abort });
7   return p;
8 }

```

- **Image staging.**

- *What.* Stage multi-image questions: load the primary image first with `fetchpriority="high"`, then secondary images; expose `sizes/srcset` in `ReliableImage`.
- *Why now.* Leverages prefetch signals and analytics from earlier weeks.
- *Impact.* Faster contentful paint and reduced bandwidth spikes.

```

1 <ReliableImage src={heroUrl} alt="Hero" loading="eager" />

```

```
2 {thumbs.map(u => <ReliableImage key={u} src={u} alt="Detail" loading="lazy" />)}
```

- **Error boundary & taxonomy.**

- *What.* Add an error boundary around the question panel and standardize error codes across save/submit/image paths to power consistent messaging and analytics.
- *Why now.* Dependent on structured errors and retry patterns from Week 2.
- *Impact.* Fewer white-screens; consistent user feedback and metrics.

```
1 function QuestionBoundary({ children }) {  
2   return <ErrorBoundary fallback={<div>Something went wrong. Try again.</div>}>{  
    children}</ErrorBoundary>;  
3 }
```

3 Question 3: Quick Wins

3.1 Quick Win 1 — Parallelize mount fetches

Change (what). Replace serial awaits in the mount effect with `Promise.allSettled` after resolving `user.id`; progressively render whichever results arrive first and show inline notices for any rejections.

Why it's a quick win. Low code surface and no schema changes; removes compounded latency immediately.

Effort. 2–4 hours.

Expected impact. Time-to-visible drops from 5–10s to ~2–3s under mock worst-case.

Verify. Compare DevTools waterfalls and user-perceived TTV under Slow 4G throttling before/after.

```
1 const { data: { user } } = await supabase.auth.getUser();  
2 const [accessR, assignR, sessionR, responsesR] = await Promise.allSettled([  
3   canStudentAccessAssignment(user.id, id),  
4   getHomeworkAssignment(id),  
5   getOrCreateHomeworkSession(user.id, id),  
6   getStudentHomeworkResponsesMap(user.id, id),  
7 ]);  
8  
9 if (accessR.status === 'fulfilled' && accessR.value) { /* set access */ }  
10 if (assignR.status === 'fulfilled' && assignR.value) { setAssignment(assignR.value); }  
11 if (sessionR.status === 'fulfilled' && sessionR.value) { setSession(sessionR.value); }  
12 if (responsesR.status === 'fulfilled') { setAnswers(responsesR.value || {}); }
```

3.2 Quick Win 2 — Visible save failure with one-tap retry

Change (what). On `saveHomeworkResponse`/submission failure, show a small banner/toast with a *Retry* action that resubmits the captured payload; keep optimistic UI but reconcile on success.

Why it's a quick win. Small UI addition; no backend work; eliminates silent failure confusion.

Effort. 1–2 hours.

Expected impact. Restores user trust and reduces lost-work reports; clear recovery path.

Verify. Force failures via mock; confirm banner appears, retry succeeds, and analytics emit success/failure events.

```
1 const pending = { studentId: user.id, qid: question.id, payload: answer };
2 const ok = await saveHomeworkResponse(user.id, question.id, assignment.id, answer,
   question, session.id);
3 if (!ok) {
4   setToast({
5     kind: 'error',
6     text: 'Save failed. Retry?',
7     retry: () => saveHomeworkResponse(pending.studentId, pending.qid, assignment.id,
   pending.payload, question, session.id)
8   });
9 }
```

3.3 Quick Win 3 — Image reliability nudge

Change (what). Nudge image start earlier by ensuring a larger look-ahead and stabilizing layout with explicit dimensions or aspect ratio at call sites.

Why it's a quick win. Pure call-site tweaks; no component overhaul; fast perceived speed gain.

Effort. 1 hour.

Expected impact. Fewer placeholder flashes; images appear closer to when users scroll into view.

Verify. Check earlier image requests in waterfalls and reduced placeholder occurrences in session captures.

```
1 <ReliableImage
2   src={imgUrl}
3   alt="Question visual"
4   className="rounded-xl"
5   style={{ aspectRatio: '4 / 3' }}
6 />
```